GenAI Intern Assignment



Objective

Build a conversational chatbot that searches, analyzes, matches, and adapts recipes from the Recipe Box dataset. The focus is reliable answers grounded in data, not plausible-sounding responses. The chatbot must handle follow-up questions and retain relevant context across turns.

Dataset and setup

Dataset: https://eightportions.com/datasets/Recipes/

- Use all three JSON files provided by the dataset. Preserve a stable recipe ID and the original title, ingredient list, and instructions for every result.
- You may use an LLM API (including the free Google AI Studio tier), a local vector index such as FAISS, SQL, or other tools. The choice of architecture is yours.
- Provide a runnable chatbot interface. A command-line interface or simple web UI is sufficient. Users should be able to ask follow-up questions in the same session.
- For dataset facts, retrieve the underlying recipes. For counts, rankings, averages, and other statistics, calculate results programmatically on the relevant data rather than asking an LLM to guess.

Task 1. Recipe search and factual Q&A (3/10)

Create a chatbot that can search the recipe collection, answer questions about specific recipes, and compute statistical answers over the dataset.

Requirements

- Support search using dish names, ingredients, and natural-language requests. Return recipe titles and stable IDs, with the relevant ingredients and instructions when requested.
- Answer follow-up questions using prior conversation context. A newly stated restriction should apply to the active search unless the user changes it.
- Support analytical questions such as recipe counts, common ingredients, average/median ingredient counts, and ranking by number of instruction steps. State the filters, number of recipes analyzed, and the calculation used.
- Ground factual claims in retrieved records. For a comparison or aggregate, use the complete filtered result set, not just a few recipes returned by semantic search.
- Do not fabricate unavailable attributes such as exact nutrition values or cooking times. If the dataset cannot support an answer, say what is missing. Clearly label any external facts or estimates if used.

Example conversations

User: Find a penne pasta recipe. User: I cannot have dairy. Expected: Refine the previous search and show compatible recipes, with evidence from their ingredient lists.
User: How many recipes contain both garlic and onion? What percentage of the dataset is that? Expected: Compute a reproducible count and percentage, identify the denominator, and explain ingredient-name normalization.
User: Among recipes containing tomato, which five have the fewest instruction steps? Expected: Rank matching recipes by a defined step-counting rule and show recipe IDs.

Task 2. Ingredient-based recipe matching (3/10)

Given a list of available ingredients and a set of criteria, retrieve and rank the closest existing recipes from the dataset. Do not invent new recipes for this task.

Requirements

- Accept ingredient names, optionally with quantities, along with hard constraints (for example, exclude peanuts) and preferences (for example, fewer missing ingredients or fewer steps).
- Normalize equivalent ingredient names where justified. Treat different forms or products as different when the distinction affects the match.
- Rank recipes using a clear, explainable scoring approach. Show matched ingredients, missing ingredients, extra required ingredients, and why each result is ranked where it is.
- Never relax a hard constraint silently. If no eligible recipe exists, state that no match was found and, separately, offer the closest alternatives only if they are clearly marked as violating a constraint.
- If quantities are provided, use them when the dataset supports a reliable comparison. Otherwise report a name-level match and state that quantity sufficiency was not verified.

Example request

User: I have eggs, onions, tomatoes, and rice. Suggest five recipes using as many of these as possible, with no peanuts and at most six instruction steps. Prefer recipes that need the fewest additional ingredients.
Expected: A ranked list of eligible recipe IDs, matched/missing ingredients for each, step counts, and a short explanation of the ranking. A follow-up such as "Exclude eggs too" should update the results.

Task 3. Evidence-based ingredient substitution (4/10)

Adapt an existing recipe when an ingredient is unavailable or restricted. Use knowledge from recipes in the local dataset as the primary source of evidence. Optional internet search is allowed only as a secondary source.

Requirements

- Given a recipe and a requested change, propose one or more substitutes that fit the user's restrictions. Consider the ingredient's role in the dish, such as flavor, moisture, thickening, binding, or texture.
- Search the local dataset first for similar recipes, ingredient variants, and relevant preparation methods. Cite the recipe IDs that support the proposed changes. Explain when the evidence is indirect.
- If local evidence is inadequate, either state the uncertainty or optionally use internet search. Clearly distinguish local recipe evidence from external sources, and include external links when used.
- Return an updated ingredient list and revised instructions, not just a list of possible substitutions. Explain likely effects on taste, texture, or preparation and distinguish predictions from verified outcomes.
- Respect hard restrictions, including stated allergies and forbidden ingredients. Do not assert that a substitution is medically safe based only on recipe text. If a reliable adaptation cannot be supported, say so.

Example conversations

User: Take this banana bread recipe and remove eggs. Use only ingredients that appear in the dataset. What replacements have evidence from related baked recipes, and how might the texture change?
User: Replace cream in this pasta recipe without dairy. Prefer examples from recipes already in the database. Expected: Show local supporting recipes, suggested changes, adjusted instructions, and expected trade-offs.

Evaluation and edge cases

- Demonstrate at least five test conversations across the three tasks, including multi-turn follow-ups, an aggregate statistics query, a no-match case, and a substitution with weak or conflicting evidence.
- Include a small reproducible evaluation set. Report what worked, what failed, and why. Show at least one case where a simple one-shot LLM answer is incorrect or ungrounded and how your system handles it.
- Clearly explain decisions around retrieval, normalization, ranking, computation, uncertainty, and use of external search. Correctly refusing an unsupported answer is better than fabricating one.

AI tools

You can use AI coding tools. Briefly disclose where and for what you've used them. You're not expected to outsource your thinking to these tools and are expected to clearly understand and explain the underlying methods used in these tasks. The initial interview will involve you walking through your code, adding modifications as suggested by your interviewer as expected.

Submission

Create a publicly accessible GitHub repo for this, along with a clear and brief README on running it in the interviewer's machine. Submit your work within 4 days of receiving the assignment. Partial submissions are acceptable.